

Mikrocontrollerfamilie 8051

Befehlssatz

Abkürzungen:

<i>A</i> :	Akkumulator
<i>Rn</i> :	Register R0..R7
<i>Ri</i> :	R0 oder R1
<i>dadr</i> :	direkte Byte-Adresse im int. Speicher (auch SF-Register)
<i>badr</i> :	direkte Bit-Adresse im int. Speicher (auch SF-Flags)
<i>adr16</i> :	absolute 16-bit Adresse
<i>adr11</i> :	11-bit Adresse relative zu 2k-Seite (0 .. 2047 bzw. 0 .. 7FFH)
<i>rel</i> :	8-bit Adresse relative zu Befehl (-128 .. +127 bzw. 80H .. 7FH)
<i>konst8</i> :	8-bit Konstante (0 .. 255 bzw. 0 .. 0FFH)
<i>konst16</i> :	16-bit Konstante (0.. 65535 bzw. 0 .. 0FFFFH)
<i>#B</i> :	Anzahl der Bytes, die der Befehl belegt
<i>#Z</i> :	Anzahl der Prozessorzyklen für die Befehlsausführung

Adressierungsarten:

Registeradressierung: in Befehlen, die nur mit *A*, *B*, *Rn* und *DPTR* arbeiten
Bsp.: `MOV A,R0`

Direkte Adressierung: alle Adressen im internen Speicher (0..7FH und SFR)
Bsp.: `MOV dadr,80H`
`MOV A,SCON`

Indirekte Adressierung (int. Speicher): Adresse (0..FF) steht in *Ri*
Bsp.: `MOV A,@R0`

Indirekte Adressierung (ext. Speicher): Adresse (0..FFFF) steht in *DPTR* oder Adresse (0..0FFH) in *Ri*
Bsp.: `MOVX A,@DPTR`

Indirekte, indizierte Adressierung (Programm-Speicher): Adresse berechnet sich aus *A* + *DPTR* oder *A* + *PC*
Bsp.: `MOVC A,@A+DPTR`

Unmittelbare Adressierung (immediate): Die Adresse ist der Befehl selbst (Laden von Konstanten)
Bsp.: `MOV R7,#0FFH` (8-bit Wert)
`MOV DPTR,#8000H` (16-bit Wert)

Transfer-Befehle (Daten kopieren):

Kopieren eines Datenbytes aus dem oder in den internen Speicher / SFR:

MOV	<i>A,Rn</i>	1 B	1 Z	<i>dadr,A</i>	2 B	1 Z
	<i>A,dadr</i>	2 B	1 Z	<i>dadr,Rn</i>	2 B	2 Z
	<i>A,@Ri</i>	1 B	1 Z	<i>dadr,dadr</i>	3 B	2 Z
	<i>A,#konst8</i>	2 B	1 Z	<i>dadr,@Ri</i>	2 B	2 Z
				<i>dadr,#konst8</i>	3 B	2 Z
	<i>Rn,A</i>	1 B	1 Z	<i>@Ri,A</i>	1 B	1 Z
	<i>Rn,dadr</i>	2 B	2 Z	<i>@Ri,dadr</i>	2 B	2 Z
	<i>Rn,#konst8</i>	2 B	1 Z	<i>@Ri,#konst8</i>	2 B	1 Z

Kopieren eines Datenbytes aus dem und in den externen Speicher:

MOVX	<i>A,@Ri</i>	1 B	2 Z	<i>@Ri,A</i>	1 B	2 Z
	<i>A,@DPTR</i>	1 B	2 Z	<i>@DPTR,A</i>	1 B	2 Z

Kopieren eines Datenbytes aus dem Programmspeicher:

MOVC	<i>A,@A+PC</i>	1 B	2 Z	
	<i>A,@A+DPTR</i>	1 B	2 Z	

Kopieren eines Datenbits von Carry in internen Datenspeicher und umgekehrt:

MOV	<i>C,badr</i>	2 B	1 Z	
	<i>badr,C</i>	2 B	2 Z	

Kopieren einer 16-bit Konstanten in das DPTR-Register:

MOV	<i>DPTR,#konst16</i>	3 B	2 Z	
------------	----------------------	-----	-----	--

Transfer-Befehle (Daten austauschen):

Austausch von Datenbytes zwischen Akkumulator und internem Speicher:

<i>XCH</i>	<i>A,Rr</i>	1 B	1 Z	
	<i>A,dadr</i>	2 B	1 Z	
	<i>A,@Ri</i>	1 B	1 Z	

Austausch der Datenbits 0..3 zwischen Akkumulator und Register:

<i>XCHD</i>	<i>A,@Ri</i>	1 B	1 Z	
--------------------	--------------	-----	-----	--

Austausch der Datenbits 0..3 und 4..7 im Akkumulator:

<i>SWAP</i>	<i>A</i>	1 B	1 Z	
--------------------	----------	-----	-----	--

Transfer-Befehle (Daten von/nach Stack kopieren):

Kopieren eines Datenbytes vom internen Datenspeicher / SFR in den Stack:

<i>PUSH</i>	<i>dadr</i>	2 B	2 Z	
--------------------	-------------	-----	-----	--

Kopieren eines Datenbytes aus dem Stack in den internen Datenspeicher / SFR:

<i>POP</i>	<i>dadr</i>	2 B	2 Z	
-------------------	-------------	-----	-----	--

Sonstige Befehle:

Leerbefehl (No operation):

<i>NOP</i>		1 B	1 Z	
-------------------	--	-----	-----	--

Logik-Befehle mit Bytes:

Das Ergebnis der logischen Operation steht immer im linken Operanden*:

ANL	<i>A,Rn</i>	1 B	1 Z	<i>dadr,A</i>	2 B	1 Z
ORL	<i>A,dadr</i>	2 B	1 Z	<i>dadr,#konst8</i>	3 B	2 Z
XRL	<i>A,@Ri</i>	1 B	1 Z			
	<i>A,#konst8</i>	2 B	1 Z			

Invertieren des Akkumulator-Inhalts:

CPL	<i>A</i>	1 B	1 Z	
------------	----------	-----	-----	--

Löschen des Akkumulator-Inhalts*:

CLR	<i>A</i>	1 B	1 Z	
------------	----------	-----	-----	--

*: Wenn das Ziel der Akkumulator ist, wird im PSW das *P*-Bit beeinflusst.

Logik-Befehle mit Bits:

Das Ergebnis der logischen Operation steht immer Carry:

ANL	<i>C,badr</i>	2 B	2 Z	
ORL	<i>C,/badr</i>	2 B	2 Z	

Invertieren eines Bits:

CPL	<i>C</i>	1 B	1 Z	
	<i>badr</i>	2 B	1 Z	

Löschen eines Bits:

CLR	<i>C</i>	1 B	1 Z	
	<i>badr</i>	2 B	1 Z	

Setzen eines Bits:

SETB	<i>C</i>	1 B	1 Z	
	<i>badr</i>	2 B	1 Z	

Arithmetik-Befehle:

Das Ergebnis der arithmetischen Operation steht immer im Akkumulator:

ADD	<i>A,Rn</i>	1 B	1 Z	
ADDC	<i>A,dadr</i>	2 B	1 Z	
SUBB	<i>A,@Ri</i>	1 B	1 Z	
	<i>A,#konst8</i>	2 B	1 Z	

Im PSW werden entsprechend dem Ergebnis die Bits *C*, *OV*, *AC* und *P* angepasst.

ADD(C): *C* = 1: Ergebnis >255

SUBB: *C* = 1: [Dest-Byte] < [Src-Byte] (z.B. [A] < konst8)

C = 0: [Dest-Byte] ≥ [Src-Byte] (z.B. [A] ≥ konst8)

Das jeweilig adressierte Datenbyte wird um 1 erhöht oder erniedrigt:

INC	<i>A</i>	1 B	1 Z	
DEC	<i>Rn</i>	1 B	1 Z	
	<i>@Ri</i>	1 B	1 Z	
	<i>dadr</i>	2 B	1 Z	

Das DPTR-Register (16 Bit) wird um 1 erhöht:

INC	<i>DPTR</i>	1 B	1 Z	
------------	-------------	-----	-----	--

Der Akkumulator *A* wird mit Register *B* multipliziert - Ergebnis in *B,A*:

MUL	<i>AB</i>	1 B	4 Z	
------------	-----------	-----	-----	--

Im PSW wird *OV* gesetzt, wenn das Ergebnis größer als 255 ist, *C* ist immer 0.

Der Akkumulator *A* wird durch Register *B* dividiert - Quotient in *A*, Rest in *B*:

DIV	<i>AB</i>	1 B	4 Z	
------------	-----------	-----	-----	--

Im PSW wird *OV* gesetzt, wenn *B* = 0 ist, *C* ist immer 0.

Dezimalkorrektur bei einer Addition von BCD-Zahlen in *A*:

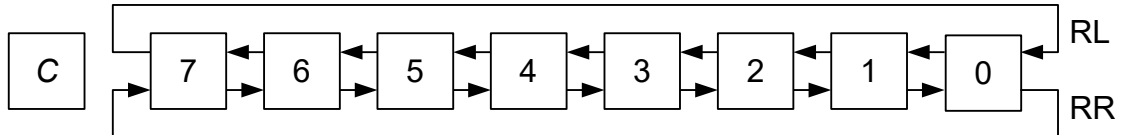
DA	<i>A</i>	1 B	1 Z	
-----------	----------	-----	-----	--

Im PSW wird *C*-Bit (Carry) angepasst.

Schiebe-Befehle

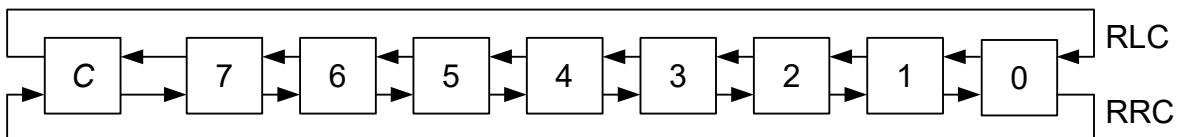
Rotiert alle Bits des Akkumulators um eins nach links oder rechts (ohne C):

<i>RL</i> <i>RR</i>	A	1 B	1 Z	
--------------------------------------	---	-----	-----	--



Rotiert alle Bits des Akkumulators mit Carry um eins nach links oder rechts:

<i>RLC</i> <i>RRC</i>	A	1 B	1 Z	
--	---	-----	-----	--



Unbedingte Sprung-Befehle:

Sprung an eine absolute 16-bit Adresse:

LJMP	<i>adr16</i>	3 B	2 Z	
-------------	--------------	-----	-----	--

Sprung an eine relative 11-bit Adresse innerhalb einer 2k-Seite:

AJMP	<i>adr11</i>	2 B	2 Z	
-------------	--------------	-----	-----	--

Sprung an eine zum Befehl relative Adresse (-128 .. +127):

SJMP	<i>rel</i>	2 B	2 Z	
-------------	------------	-----	-----	--

Sprung an eine indirekte und indizierte Adresse:

JMP	<i>@A+DPTR</i>	1 B	2 Z	
------------	----------------	-----	-----	--

Unbedingter Sprung in eine Unteroutine:

Sprung an eine absolute 16-bit Adresse:

LCALL	<i>adr16</i>	3 B	2 Z	
--------------	--------------	-----	-----	--

Sprung an eine relative 11-bit Adresse innerhalb einer 2k-Seite:

ACALL	<i>adr11</i>	2 B	2 Z	
--------------	--------------	-----	-----	--

Rücksprung aus einer Unteroutine:

Rücksprung aus einer normalen Unteroutine:

RET		1 B	2 Z	
------------	--	-----	-----	--

Rücksprung aus einer Interruptroutine:

RETI		1 B	2 Z	
-------------	--	-----	-----	--

Bedingte Sprung-Befehle:

Bedingter relativer Sprung in Abhängigkeit des Carry-Bits:

JC JNC	<i>rel</i>	2 B	2 Z	
-------------------------	------------	-----	-----	--

Bedingter relativer Sprung in Abhängigkeit vom Akkumulatorinhalt (=0 oder ≠0):

JZ JNZ	<i>rel</i>	2 B	2 Z	
-------------------------	------------	-----	-----	--

Bedingter relativer Sprung in Abhängigkeit von einem Bit:

JB JNB JBC	<i>badr,rel</i>	3 B	2 Z	
---------------------------------------	-----------------	-----	-----	--

Bedingter relativer Sprung, wenn Akkumulator ungleich Datenbyte oder Akkumulator/Register/Datenbyte ungleich Konstante ist:

CJNE	<i>A,dadr,rel</i>	3 B	2 Z	
	<i>A,#konst8,rel</i>	3 B	2 Z	
	<i>Rn,#konst8,rel</i>	3 B	2 Z	
	<i>@Ri,#konst8,rel</i>	3 B	2 Z	

C = 1: [Dest-Byte] < [Src-Byte] (z.B. [A] < konst8)

C = 0: [Dest-Byte] ≥ [Src-Byte] (z.B. [A] ≥ konst8)

Das Register/Datenbyte wird um 1 erniedrigt, bedingter relativer Sprung, wenn ≠0:

DJNZ	<i>Rn,rel</i>	2 B	2 Z	
	<i>dadr,rel</i>	3 B	2 Z	